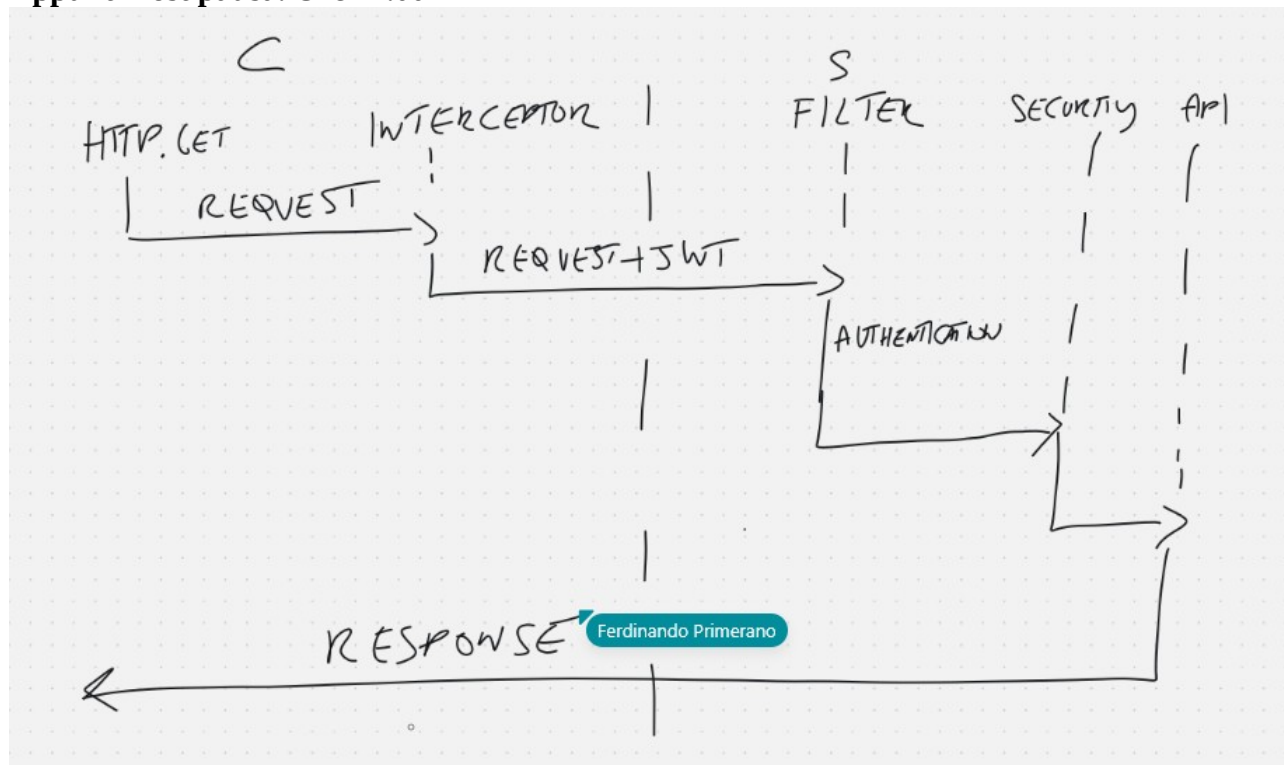




. Il diagramma descrive il flusso di comunicazione tra un client e un server, con particolare attenzione alla gestione della sicurezza tramite JWT (JSON Web Token). In termini tecnici, questo schema rappresenta il pattern Interceptor-Filter per l'autenticazione in un'architettura REST.

Lato Client: l'interceptor

Il processo inizia quando l'applicazione — ad esempio un'app Angular, React o Android — deve inviare una richiesta al server. Il client genera la richiesta HTTP, ma prima che essa lasci effettivamente il dispositivo, passa attraverso un interceptor. Il suo compito è iniettare automaticamente le credenziali, ovvero il JWT, nell'header della richiesta, solitamente sotto la chiave `Authorization: Bearer <token>`. La richiesta esce quindi dal client arricchita con il token di sicurezza.

Lato Server: la catena di filtri

Una volta che la richiesta arriva al backend, non raggiunge immediatamente i dati, ma deve superare una serie di controlli. Il filtro è il primo punto di ingresso: legge l'header della richiesta per verificare se il JWT è presente e strutturalmente valido. Successivamente avviene la fase di autenticazione, in cui il server convalida la firma del token; se il token è valido e non è scaduto, l'utente viene considerato autenticato. Si passa poi alla fase di autorizzazione, in cui il sistema verifica se l'utente autenticato possiede i permessi, ovvero i ruoli, necessari per accedere alla risorsa specifica richiesta.

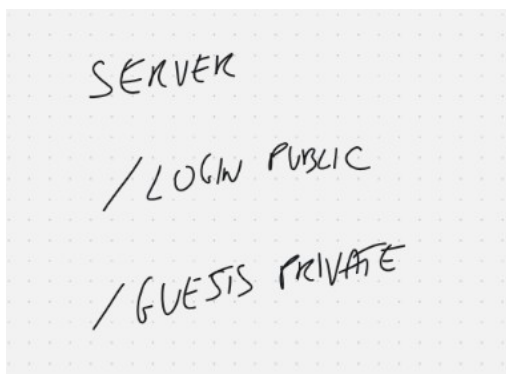
Elaborazione e risposta

Solo se tutti i passaggi precedenti hanno avuto esito positivo, la richiesta raggiunge l'endpoint dell'API, ovvero il controller, che elabora i dati o interroga il database. Il server invia quindi il risultato al client, sotto forma di dati richiesti o messaggio di conferma. Se il JWT fosse assente o non valido, il filtro bloccherebbe la richiesta immediatamente, restituendo un errore 401 Unauthorized senza coinvolgere in alcun modo l'API.

Perché questo modello è rilevante

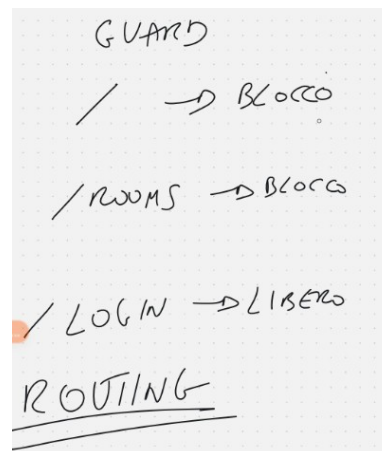
Il server non ha bisogno di sessioni né di salvare i dati dell'utente in memoria, poiché tutte le informazioni necessarie sono contenute all'interno del JWT inviato dal client: il sistema è quindi stateless per definizione. Questo approccio garantisce inoltre una netta separazione delle responsabilità, poiché l'API non si occupa di verificare l'identità del chiamante — questo lavoro viene svolto a monte dai filtri di sicurezza. Grazie all'interceptor, infine, lo sviluppatore non deve aggiungere manualmente il token ad ogni singola chiamata nel codice, riducendo così il rischio di errori e rendendo il processo completamente automatico.

In angular posso dire ad una pagina, non puoi essere caricato se non sei loggato. Questo controllo non serve per dare sicurezza ma per aiutare l'utente. E' aggirabile. La Guard



SERVER

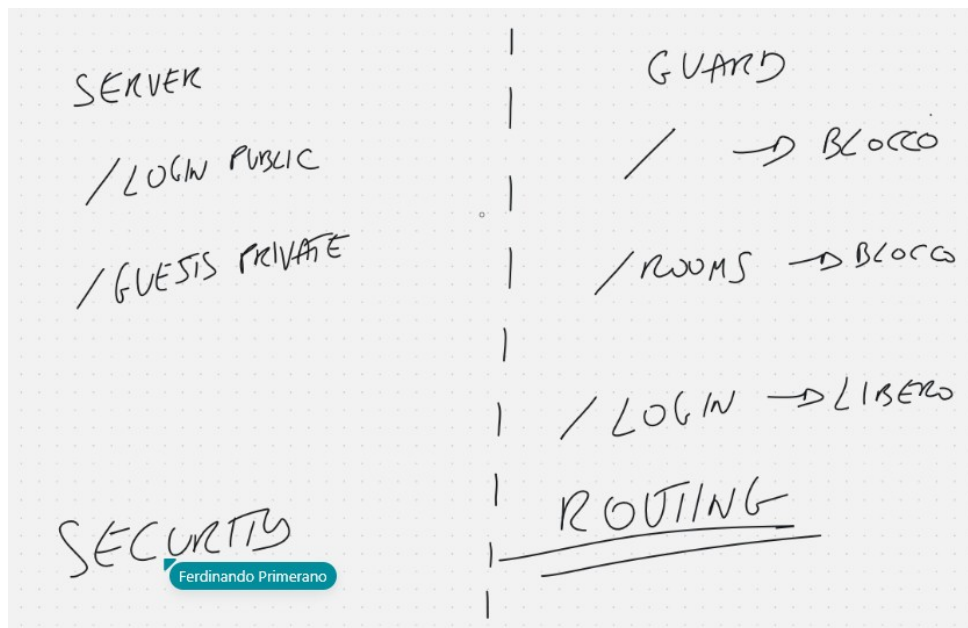
- /LOGIN PUBLIC
- /GUESTS PRIVATE



GUARD

- / → BLOCCO
- /rooms → BLOCCO
- /LOGIN → LIBERO

ROUTING



1. Il Ruolo della "Guard" (CanActivate)

In Angular, una **Guard** è un pezzo di logica che si mette "davanti" a una rotta. Quando l'utente clicca su un link (es. `/rooms`), Angular non carica immediatamente il componente, ma chiede alla Guard: "L'utente può passare?".

- **/login -> LIBERO:** La Guard vede che la rotta è pubblica, restituisce `true` e Angular carica il modulo di login.
- **/rooms o / -> BLOCCO:** La Guard controlla se esiste un token valido (JWT) nel `localStorage`. Se non c'è, restituisce `false`.
- **Risultato:** La navigazione viene annullata. L'utente rimane dove si trova oppure, come succede quasi sempre, viene reindirizzato forzatamente alla pagina di `/login`.

Caratteristica	Client-Side (Angular Guard)	Server-Side (Security/API)
Obiettivo	User Experience (UX). Evita che l'utente veda una pagina vuota o rotta se non ha i permessi.	Integrità dei Dati. Impedisce l'accesso reale ai dati nel database.
Azione	Blocca il rendering del componente o reindirizza al login.	Blocca la richiesta HTTP con un errore <code>401 Unauthorized</code> .
Sicurezza	Facilmente aggirabile da un utente esperto (può modificare il codice JS nel browser).	Invalicabile (se ben configurata), perché il server non si fida del client.

Il flusso tipico è:

1. L'utente prova ad andare su `/guests` (Private).
2. La Guard `AuthGuard` interviene.
3. Se l'utente non è loggato, la Guard esegue `this.router.navigate(['/login'])`.

4. L'utente vede la pagina di login senza che l'intera pagina del browser si sia mai ricaricata.

Consideriamo il progetto superiorB&B

il lato server è già protetto, se anche lasciassimo il componente libero, il server si rifiuterebbe comunque, ma non è bello.

```
TS user-service.ts  TS room-service.ts M  TS authInterceptors.ts M  <> top-menu.html  <> room-manag

src > app > TS app.routes.ts > routes > component
1  import { Routes } from '@angular/router';
2  import { BookRoom } from '../book-room/book-room';
3  import { FormLogin } from '../form-login/form-login';
4  import { authGuard } from '../guards/auth.guard';
5  import { RoomManagement } from '../room-management/room-management';
6
7  export const routes: Routes = [
8    { path: '', component: BookRoom, canActivate: [authGuard]},
9    { path: 'login', component: FormLogin},
10   { path: 'rooms', component: RoomManagement}
11 ];
12
```

Se non sono un receptionist non dev poter andare sul room manager ad esempio.

```
src > app > TS app.routes.ts > routes > canActivate
1  import { Routes } from '@angular/router';
2  import { BookRoom } from '../book-room/book-room';
3  import { FormLogin } from '../form-login/form-login';
4  import { authGuard } from '../guards/auth.guard';
5  import { RoomManagement } from '../room-management/room-management';
6
7  export const routes: Routes = [
8    { path: '', component: BookRoom, canActivate: [authGuard]},
9    { path: 'login', component: FormLogin},
10   { path: 'rooms', component: RoomManagement, canActivate: [authGuard]}
11 ];
12
```

Quindi gli dico che ha un guardiano.

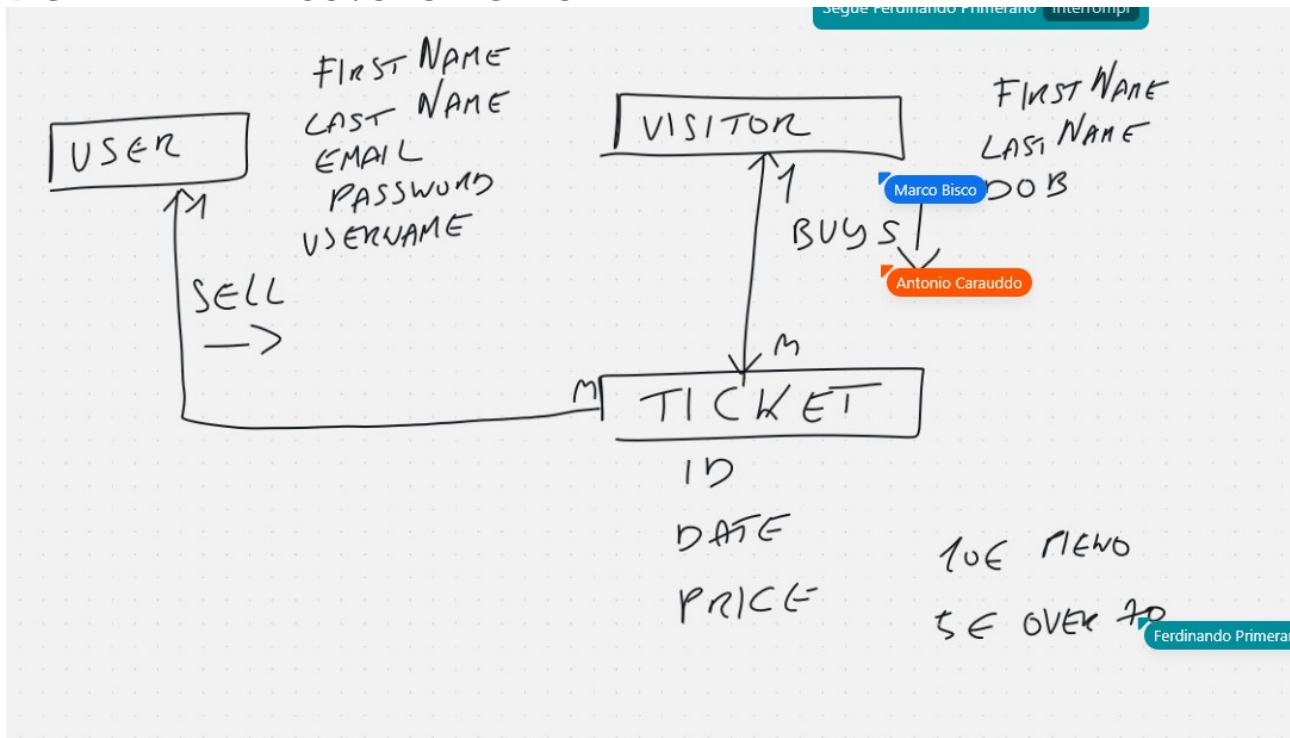
```
EXPLORER    ...  ce.ts  TS room-service.ts M  TS authInterceptors.ts M  top-menu.html  room-management.html U  TS app.routes.ts M  TS auth.guard.ts M X  TS book-ro...  ...

SUPERIORBB
  public
  src
    app
      book-room
        # book-room.css
        book-room... M
        TS book-room.ts
      form-login
      guards
      TS auth.guard.ts M
      guest-picker
      home-page
      insert-quick-booking
      interceptors
      login-test
      model
      ...

src > app > guards > TS auth.guard.ts > authGuard
1 import { inject } from '@angular/core';
2 import { CanActivateFn, Router } from '@angular/router';
3
4 export const authGuard: CanActivateFn = (route, state) => {
5   const router = inject(Router);
6   const token = localStorage.getItem('token');
7
8   if (token) {
9     return true;
10  }
11
12  // RIDIREZIONO alla pagina di login se non trovo il token
13  return router.createUrlTree(['/login']);
14 };
15
```

Le guardie non sono sicurezza, sono regole per capire se posso attivare una rotta oppure no.

BIGLIETTERIA – NUOVO ESERCIZIO



API SPRINT BOOT FULL CRUD +
USERS/LOGIN (JWT TOKEN)

CV (GUARD) NEW GUEST NEW TICKET TICKET SEARCH

LOGIN

USERNAME
 PASSWORD

Ferdinando Primerano

CV (GUARD) NEW GUEST NEW TICKET TICKET SEARCH

FIRST NAME
 LAST NAME
 DOB
 SSN

1 GUARD
1 INTERCEPT

2° NEW GUEST

Ferdinando Primerano

Lato server studiare pacchetto security.

SpringBoot – Database , rapporti, entità e tutto quello che ne compete.

Appena fatto simuliamo il login da postman come prima.

- 1 BACKEND OPEN
- 2 ALL SECURITY
- 3 TEST POSTMAN